Anti-Bluff Mandate Reinforcement — Group B Design

> **Status:** Design approved (operator, 2026-05-05 evening) after Group A-prime closure. Implementation pending — subagent-driven execution per Group A-prime pattern. Branch: Containers `lava-pin/2026-05-06-group-b`; Lava parent commits on `master` once the pin lands.

> **Forensic anchor.** Group A-prime mechanically closed §6.N-debt (pre-push Check 4 + 5, `check-constitution.sh` §6.N awareness, `cmd/emulator-cleanup`, per-AVD gradle stdout persistence). What it left open is *the next class of bluffs the matrix gate cannot yet detect*: matrix-runner reliability gaps (zombie cleanup is best-effort but Teardown still relies on the upstream `adb emu kill` that has been observed to hang), evidence opacity (`real-device-verification.md` rows lack the `target/sdk/device` triple a reviewer needs to verify "the AVD that ran is the AVD the matrix claims it ran"), JUnit-XML evidence ambiguity (a failed test row only carries `passed: false` and a JUnit path — the reviewer must open the file to learn *which* test failed and *why*), and dev/CI conflation (`--concurrent N` does not exist; iteration runs and gating runs share the same evidence shape).
>
> Group B is the next reinforcement increment. Five tightenings, each independently shippable, all bound to the same anti-bluff mechanic the rest of §6.A-§6.N enforce: the gate must distinguish "feature works for a real user on the gating matrix" from any weaker signal, and the evidence row must carry enough detail that a future reviewer cannot rationalize past a divergence.

1. Scope

1.1 In scope (5 components, locked)

1. **Per-row diagnostic data in attestation JSON.** Every row in `real-device-verification.{json,md}` (clause 6.I clause 4) gains a `diag` object: `target` (full AVD `system-images` package id), `sdk` (the AVD's reported `ro.build.version.sdk`), `device` (the AVD's

reported `ro.product.model` / `ro.product.device`), `adb_devices_state` (the line for this emulator from `adb devices -l` captured immediately before instrumentation invocation). This is the minimum forensic surface a reviewer needs to verify the AVD the matrix *claims* to have run is the AVD that *actually* ran.

2. **Teardown force-kill fast-path.** `pkg/emulator.AndroidRunner.Teardown` today calls `adb -s <serial> emu kill` then waits up to 30s for the emulator to exit (observed to hang on stuck QEMU instances; the stuck instance becomes a zombie that `cmd/emulator-cleanup` then has to deal with on the *next* matrix run). Group B adds a port-targeted fast-path: after the 30s grace expires, `Teardown` invokes `cleanup.KillByPort(port)` which walks `/proc`, finds the QEMU process whose cmdline contains `-port <port>` *as adjacent argv tokens* (strict — substring match is forbidden because port `5554` substring- matches `25554`), SIGTERMs it, waits 5s, SIGKILLs if still alive. **Safety: if no `/proc` entry has `-port <port>` adjacent, the fast-path is a no-op and Teardown returns the original `adb emu kill` outcome.** The kill is *only* applied to a process whose cmdline is provably this matrix's emulator; no PID file scraping, no name-prefix match alone, no broader pkill. Other concurrent emulators on other ports — whether started by another matrix run, by a developer's manual `emulator -avd`, or by a sibling vasic-digital project — are untouched. This is the "safest approach" the operator selected when offered three alternatives.

3. **Pack-dir convention wiring.** `scripts/run-emulator-tests.sh` today accepts `--evidence-dir <path>` with a default of `.lava-ci-evidence/<UTC>-matrix`. Group B adds a Lava-side wrapper convention: the script auto-detects `versionName` and `versionCode` from `app/build.gradle.kts` and constructs the evidence path as `.lava-ci-evidence/Lava-Android-<versionName>-<versionCode>/matrix/<UTC>/`. `--tag <tag>` overrides the `Lava-Android-<v>-<vc>` prefix verbatim (used by `scripts/tag.sh` to anchor evidence under the exact tag being cut). `--evidence-dir` (the existing flag) still wins if both are passed — the override layer is `--evidence-dir` > `--tag` > auto-detect. The Containers `cmd/emulator-matrix` binary is *not* changed; the convention is Lava-domain (which version string, which tag prefix) and stays in the Lava-side bash wrapper per Decoupled Reusable Architecture.

4. **JUnit XML failure parsing → `failure_summaries[]` in attestation row.** `pkg/emulator.AndroidRunner` already copies the JUnit XML report into `<EvidenceDir>/<avd.Name>/test-report/` (Group A-prime). Group B parses each XML file post-test, extracts every `<failure>` and `<error>` element's `message` attribute and full text content, and writes a `failure_summaries: [{class, name, type, message, body}]` array on the row. Empty array on

success; missing `<failure>+<error>` is success; parse error is recorded as a synthetic `{class:"<unparseable>", message:"junit-xml parse failed: <err>"}` entry but does NOT fail the row (the JUnit XML is evidence, not the gating signal — Sixth Law clause 3 puts the gating signal on the user-visible test outcome, which is `passed: bool`).

5. **Concurrent-mode opt-in with gating-flip.** `cmd/emulator-matrix` gains two flags:
    - `--concurrent N` (default `1`): worker pool of N goroutines pulling from the AVD list. Each worker boots its own emulator on a unique console port (5554, 5556, 5558, ...), runs the test, tears down. Default 1 preserves today's serial behavior.
    - `--dev`: developer-iteration mode. Implies `--no-cold-boot` is OK, implies snapshot reload is OK, implies the row may carry an unbooted emulator timeout without re-attempting. Convenience for fast iteration. Either flag (`--concurrent != 1` OR `--dev`) sets `gating: false` on the run-level attestation field. `gating: true` is the clause-6.I- clause-7-acceptable signal; `scripts/tag.sh` MUST refuse any commit whose `real-device-verification.json` carries `gating: false` or whose per-row `diag.sdk` does not equal the row's `api_level`. The flag defaults preserve today's gating behavior — a developer who runs the matrix the way Group A-prime ships gets `gating: true` for free.

1.2 Out of scope (deferred to later groups)

- Network simulation / cellular profile per AVD (Group C territory).
- Hardware-screenshot capture per row (Group C — separate evidence pipeline, separate disk-pressure analysis).
- AVD download / image-cache management (Containers concern; out of Lava matrix-runner scope).
- Multi-host worker distribution (Containers `cmd/distributed-test` territory; the current matrix runner is single-host by design).
- iOS / non-Android emulator support (6.K-debt roadmap item).

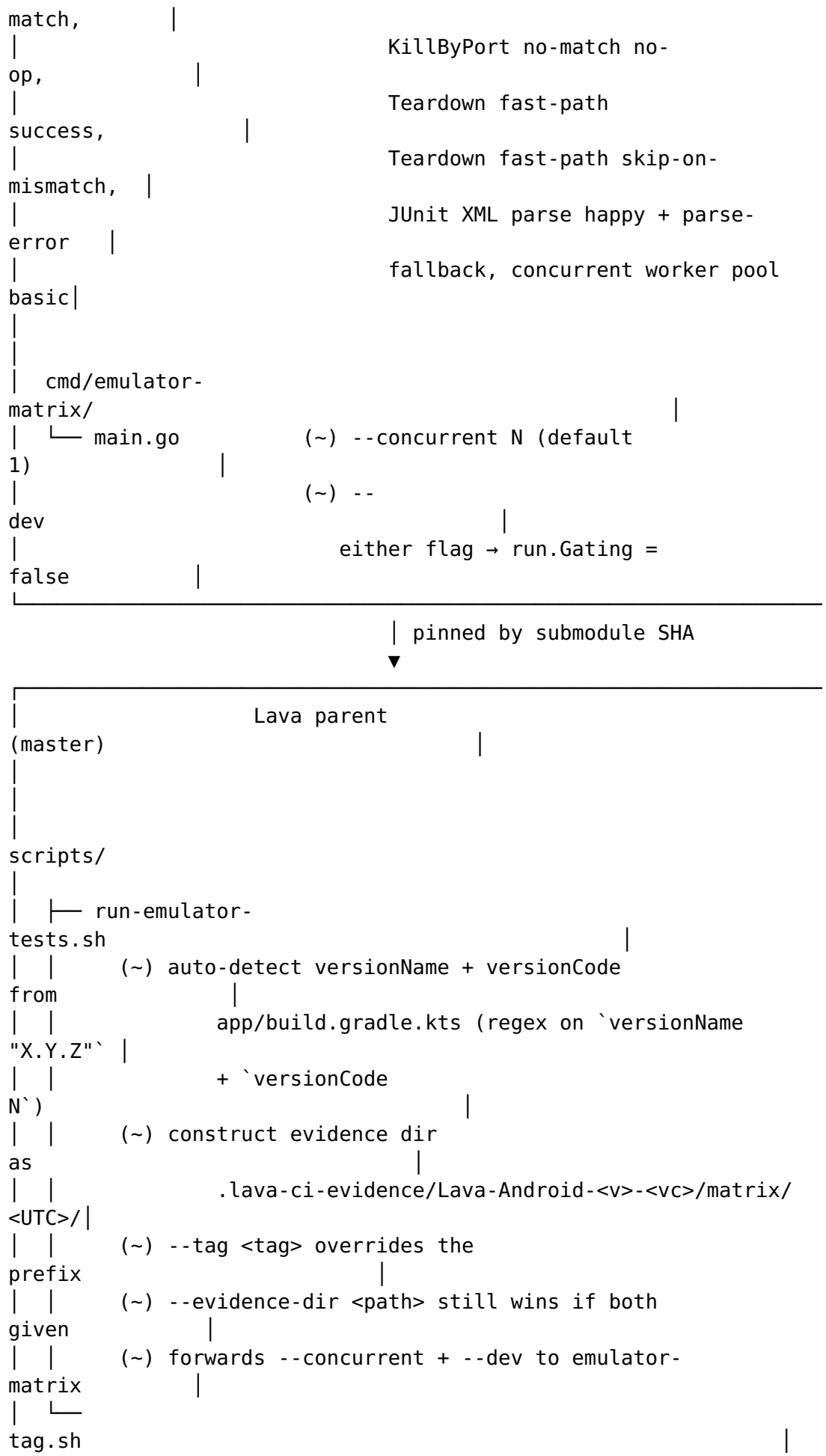
1.3 Non-goals

- **Not** a re-design of the matrix runner. Group B layers reliability
    - observability fixes onto the Group A-prime architecture; the per-AVD attestation row, the cold-boot mandate, the `gating: true` default, the 4-mirror push gate all stay.
- **Not** a relaxation of any clause. Every Group B addition tightens the gate (Teardown is more reliable, evidence row carries more detail, developer-mode is opt-in and self-flags as non-gating). Nothing weakens the existing gating signal.

- **Not** a Lava-only change. Components A, B, C, D, E live in the Containers submodule because they are matrix-runner reliability and evidence concerns that the next vasic-digital project to use the emulator package would want identically. Components F, G live in the Lava parent because the version-string auto-detect and the `tag.sh` gate are Lava-domain (per Decoupled Reusable Architecture).

2. Architecture

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                   Containers submodule                                       │
│                   (branch lava-pin/2026-05-06-group-b)                       │
│                                                                              │
│  pkg/emulator/                                                               │
│  ├── cleanup.go       (+) KillByPort(port) (KillReport, error)               │
│  │                        strict /proc walk, "-port N" adjacent             │
│  │                        token match; SIGTERM 5s SIGKILL                   │
│  ├── android.go       (~) Teardown: after 30s adb-emu-kill grace,           │
│  │                        invoke KillByPort(port); skip on                  │
│  │                        mismatch; log outcome to row                      │
│  ├── types.go         (+) DiagnosticInfo struct                             │
│  │                    (+) KillReport struct                                 │
│  │                    (~) AVDRow.Diag, FailureSummaries, Concurrent         │
│  │                    (~) RunReport.Gating                                  │
│  ├── matrix.go        (~) Capture diag pre-test (target via avd             │
│  │                        manager, sdk via getprop, device via              │
│  │                        getprop, adb_devices_state via adb                │
│  │                        devices -l). Parse JUnit XML post-test.           │
│  │                        Optional worker pool when concurrent>1.           │
│  └── *_test.go        (+) Tests for KillByPort strict-
```


match,
KillByPort no-match no-op,
Teardown fast-path success,
Teardown fast-path skip-on-mismatch,
JUnit XML parse happy + parse-error
fallback, concurrent worker pool basic
cmd/emulator-matrix/
└── main.go
(~) --concurrent N (default 1)
(~) --dev
either flag → run.Gating = false
pinned by submodule SHA
Lava parent (master)
scripts/
├── run-emulator-tests.sh
(~) auto-detect versionName + versionCode from app/build.gradle.kts (regex on `versionName "X.Y.Z"` + `versionCode N`)
(~) construct evidence dir as .lava-ci-evidence/Lava-Android-<v>-<vc>/matrix/<UTC>/
(~) --tag <tag> overrides the prefix
(~) --evidence-dir <path> still wins if both given
(~) forwards --concurrent + --dev to emulator-matrix
└── tag.sh


(~) gate 1: reject if attestation has concurrent != 1
(~) gate 2: reject if attestation has gating: false
(~) gate 3: assert per-row diag.sdk == row.api_level
– divergence rejects the tag
tests/
tag-helper/
test_tag_rejects_concurrent_attestation.sh
test_tag_rejects_non_gating_attestation.sh
test_tag_rejects_diag_sdk_mismatch.sh
test_tag_accepts_gating_serial_attestation.sh
pre-push/
(no new files; Group A-prime Check 4 already covers changes to scripts/run-emulator-tests.sh + scripts/tag.sh)
.lava-ci-evidence/
bluff-hunt/2026-05-06-group-b-evening.json
Phase-Group-B-closure-2026-05-06-evening.json
CLAUDE.md
(~) §6.I clause 4 – extend AVDRow row schema documentation with diag + failure_summaries + concurrent fields

3. Components

A. `KillByPort` in `pkg/emulator/cleanup.go` (Containers)

Signature:

```go
// KillByPort attempts to terminate the QEMU process whose cmdline
        contains
// "-port <port>" as adjacent argv tokens. It is strict by
        construction:
//
//   - substring match is forbidden — "-port 5554" must NOT match
        a process
//     whose argv contains "-port" "25554" or "55540"
//   - if no /proc entry has the adjacent token pair, KillByPort
        is a no-op
//     and reports KillReport{Matched: 0}
//   - the matched process receives SIGTERM, then up to 5s grace,
        then
//     SIGKILL if still alive
//   - the procWalker + killer interface seams from Cleanup() are
        reused
//
// Concurrent emulators on other ports are NOT touched. Other
        vasic-digital
// projects or developer-spawned QEMU instances on other ports are
        NOT
// touched. The 1-process-1-port invariant is the safety property.
func KillByPort(ctx context.Context, port int) (KillReport, error)

// killByPortWithDeps is the testable core; production KillByPort
        wires the
// real procWalker + killer.
func killByPortWithDeps(ctx context.Context, port int, w
        procWalker, k killer) (KillReport, error)
```

Strict matching algorithm:

```
for each /proc/<pid>:
  read /proc/<pid>/cmdline (NUL-separated argv)
  split into argv slice
  for i := 0; i < len(argv)-1; i++:
    if argv[i] == "-port" && argv[i+1] == strconv.Itoa(port):
      MATCH — record pid
      break inner loop
  (no match — skip this pid)

if matched.empty():
  return KillReport{Matched: 0}, nil  # no-op safe

for each matched pid:
  send SIGTERM via killer
poll up to 5s for /proc/<pid> disappearance
```

```
for each still-alive pid:
  send SIGKILL via killer
return KillReport{Matched: len(matched), Sigtermed: ...,
Sigkilled: ...}
```

Why strict adjacency. A naive `strings.Contains(cmdline, "-port 5554")` matches `-port 25554` because the cmdline is NUL-separated and the substring search ignores word boundaries. A naive `regexp.MustCompile(\b-port 5554\b)` matches the same because `\b` does not honour QEMU's argv tokenization. The only correct implementation is to split on NUL and compare adjacent tokens — which is what the production code does. The accompanying `TestKillByPort_StrictMatch` test covers exactly this case (it injects a procWalker that returns a process whose cmdline contains `-port 25554` and asserts `Matched == 0`).

B. `Teardown` fast-path in `pkg/emulator/android.go` (Containers)

Today (Group A-prime end state):

```go
func (r *AndroidRunner) Teardown(ctx context.Context, avd AVD)
        error {
  // best-effort: adb -s <serial> emu kill
  // wait up to 30s for the emulator to exit
  // return the wait outcome
}
```

Group B extension:

```go
func (r *AndroidRunner) Teardown(ctx context.Context, avd AVD)
        error {
  // existing: adb -s <serial> emu kill, wait up to 30s
  if exited := waitForExit(ctx, avd, 30*time.Second); exited {
    return nil
  }

  // fast-path: KillByPort on the AVD's console port
  report, err := KillByPort(ctx, avd.ConsolePort)
  if err != nil {
    // forensic-only: log, do not fail Teardown
    r.log.Warnf("Teardown KillByPort failed for %s on port %d:
        %v",
                avd.Name, avd.ConsolePort, err)
  }
  if report.Matched == 0 {
    // safety: no process matched the strict criterion → fall back
        to
    // the adb-emu-kill outcome
    return errors.New("teardown timeout, KillByPort matched 0
        processes")
  }
```

```
  // KillByPort handled it; re-poll briefly for /proc clearing
  if exited := waitForExit(ctx, avd, 5*time.Second); exited {
    return nil
  }
  return
        errors.New("teardown timeout, KillByPort ran but emulator
        still present")
}
```

Safety invariants:

1. KillByPort is the second resort, not the first. The upstream `adb emu kill` retains its 30-second window — a healthy emulator exits cleanly there.
2. KillByPort is strict — no port-suffix collision risk.
3. KillByPort skips on mismatch — concurrent emulators / sibling-project QEMUs are untouched.
4. The Teardown error is preserved — a stuck emulator that the fast-path could not even *find* still surfaces as a Teardown failure (the matrix runner already records this as a row failure).

C. `DiagnosticInfo` + `KillReport` in `pkg/emulator/types.go` (Containers)

```
// DiagnosticInfo is the per-AVD forensic snapshot captured
        immediately
// before instrumentation invocation. Reviewer-facing — answers
        "is the AVD
// the matrix claims it ran the AVD that actually ran?".
type DiagnosticInfo struct {
  Target          string `json:"target"`             // system-
        images package id, e.g. "system-
        images;android-34;google_apis;x86_64"
  SDK             int    `json:"sdk"`                //
        ro.build.version.sdk
  Device          string `json:"device"`             //
        ro.product.model || ro.product.device
  ADBDevicesState string `json:"adb_devices_state"`  // line from
        `adb devices -l` for this emulator
}

// FailureSummary is one parsed JUnit <failure> or <error> entry.
type FailureSummary struct {
  Class   string `json:"class"`
  Name    string `json:"name"`
  Type    string
        `json:"type"`              // "failure" | "error" |
        "<unparseable>"
  Message string `json:"message"`           // <failure
        message="..."> attribute
```

```
  Body    string
        `json:"body,omitempty"`    // text content of the
        <failure> element
}

// KillReport is what cleanup.KillByPort returns to its caller.
type KillReport struct {
  Matched   int
        `json:"matched"`           // # of /proc entries matched
  Sigtermed []int `json:"sigtermed,omitempty"`
  Sigkilled []int
        `json:"sigkilled,omitempty"` // ones that needed SIGKILL
        after the 5s grace
}

// AVDRow extension (Group B fields appended; existing fields
        unchanged):
type AVDRow struct {
  // ... existing fields (Name, APILevel, FormFactor, Passed,
        GradleLogPath,
  //     TestReportDir, BootDuration, TestDuration, Timestamp) ...
  Diag             DiagnosticInfo   `json:"diag"`
  FailureSummaries []FailureSummary `json:"failure_summaries"` //
        [] on pass
  Concurrent       int              `json:"concurrent"`        //
        1 in serial mode
}

// RunReport extension (run-level field):
type RunReport struct {
  // ... existing fields ...
  Gating bool `json:"gating"` // false if --concurrent != 1 OR --
        dev
}
```

D. Diagnostic capture + JUnit XML parse + worker pool in `pkg/emulator/matrix.go` (Containers)

Diagnostic capture (per AVD, pre-test):

```
func (r *AndroidMatrixRunner) captureDiagnostic(ctx
        context.Context, avd AVD) DiagnosticInfo {
  d := DiagnosticInfo{}
  // adb -s <serial> shell getprop ro.build.version.sdk
  if sdk, err := r.adb.Getprop(ctx, avd.Serial,
        "ro.build.version.sdk"); err == nil {
    d.SDK, _ = strconv.Atoi(strings.TrimSpace(sdk))
  }
  // adb -s <serial> shell getprop ro.product.model (fallback
        ro.product.device)
  if model, err := r.adb.Getprop(ctx, avd.Serial,
        "ro.product.model"); err == nil &&
        strings.TrimSpace(model) != "" {
```

```
    d.Device = strings.TrimSpace(model)
  } else if dev, err := r.adb.Getprop(ctx, avd.Serial,
        "ro.product.device"); err == nil {
    d.Device = strings.TrimSpace(dev)
  }
  // avdmanager list avd → find the entry whose Name == avd.Name →
        Target field
  if target, err := r.avdmanager.GetTarget(ctx, avd.Name); err ==
        nil {
    d.Target = target
  }
  // adb devices -l → grep for avd.Serial → record full line
  if line, err := r.adb.DevicesLine(ctx, avd.Serial); err == nil {
    d.ADBDevicesState = line
  }
  return d
}
```

JUnit XML parse (post-test, every run):

```
func parseJUnitFailures(xmlPath string) []FailureSummary {
  data, err := os.ReadFile(xmlPath)
  if err != nil {
    return []FailureSummary{{
      Class: "<unparseable>", Type: "<unparseable>",
      Message: "junit-xml read failed: " + err.Error(),
    }}
  }
  // walk every <testsuite>/<testcase>; for each <testcase> with a
  // <failure> or <error> child, append a FailureSummary
  // ... (encoding/xml decoder) ...
  return summaries  // empty slice on all-pass
}
```

The parser MUST be tolerant of:

- Multiple `<testsuite>` siblings (Gradle's per-class XML output).
- `<testcase>` with both `<failure>` and `<error>` (record both).
- `<failure>` with no `message` attribute (use empty string).
- `<failure>` with no text content (empty `Body`).
- Malformed XML (return the synthetic `<unparseable>` entry — does NOT fail the row).

Worker pool (concurrent mode, opt-in):

```
func (r *AndroidMatrixRunner) Run(ctx context.Context)
        (RunReport, error) {
  rows := make([]AVDRow, 0, len(r.AVDs))
  if r.Concurrent <= 1 {
    // existing serial path, unchanged
    for _, avd := range r.AVDs {
      rows = append(rows, r.runOne(ctx, avd))
    }
```

```
  } else {
    // concurrent: bounded worker pool
    queue := make(chan AVD)
    results := make(chan AVDRow, len(r.AVDs))
    var wg sync.WaitGroup
    for i := 0; i < r.Concurrent; i++ {
      wg.Add(1)
      go func(workerIdx int) {
        defer wg.Done()
        for avd := range queue {
          // assign a unique console port per worker:
          //   port = 5554 + 2*workerIdx
          avd.ConsolePort = 5554 + 2*workerIdx
          results <- r.runOne(ctx, avd)
        }
      }(i)
    }
    for _, avd := range r.AVDs { queue <- avd }
    close(queue)
    wg.Wait()
    close(results)
    for row := range results { rows = append(rows, row) }
  }
  return RunReport{
    Rows:   rows,
    Gating: r.Concurrent == 1 && !r.Dev,
  }, nil
}
```

The 1-process-1-port invariant guarantees `KillByPort(workerPort)` in Teardown only matches the worker's own emulator.

E. `--concurrent` + `--dev` flags in `cmd/emulator-matrix/main.go` (Containers)

```
var (
  // ... existing flags ...
  concurrent = flag.Int("concurrent", 1,
    "max concurrent emulators (default 1; >1 sets gating=false)")
  dev = flag.Bool("dev", false,
    "developer iteration mode; permits snapshot reload, sets
        gating=false")
)
// ...
runner := &emulator.AndroidMatrixRunner{
  // ... existing fields ...
  Concurrent: *concurrent,
  Dev:        *dev,
}
```

The flags propagate to `RunReport.Gating` via the runner's `Gating: r.Concurrent == 1 && !r.Dev` calculation. The CLI does NOT need to print the gating flag at the top — the JSON evidence file is the source of truth, and `tag.sh` reads it from there.

F. Evidence-path auto-detection in `scripts/run-emulator-tests.sh` (Lava)

Auto-detect `versionName` + `versionCode` from `app/build.gradle.kts`:

```
detect_version() {
  local f="$PROJECT_DIR/app/build.gradle.kts"
  [[ -f "$f" ]] || return 1
  # versionName "X.Y.Z" — handle single + double quotes, leading
        whitespace
  local v
  v=$(grep -oE 'versionName[[:space:]]+"[^"]+"' "$f" | head -n1 |
        sed 's/.*"\([^"]*\)"/\1/')
  [[ -n "$v" ]] || return 1
  local vc
  vc=$(grep -oE 'versionCode[[:space:]]+[0-9]+' "$f" | head -n1 |
        awk '{print $NF}')
  [[ -n "$vc" ]] || return 1
  echo "Lava-Android-${v}-${vc}"
}

# evidence-path resolution priority:
#   1. --evidence-dir <path>   (existing flag, wins)
#   2. --tag <tag>             (new; <tag>/matrix/<UTC>/)
#   3. auto-detect             (Lava-Android-<v>-<vc>/matrix/
        <UTC>/)
if [[ -n "${EVIDENCE_DIR_OVERRIDE:-}" ]]; then
  EVIDENCE_DIR="$EVIDENCE_DIR_OVERRIDE"
elif [[ -n "${TAG_OVERRIDE:-}" ]]; then
  EVIDENCE_DIR=".lava-ci-evidence/${TAG_OVERRIDE}/matrix/$(date
        -u +%Y-%m-%dT%H-%M-%SZ)"
else
  prefix=$(detect_version) || prefix="Lava-Android-unknown"
  EVIDENCE_DIR=".lava-ci-evidence/${prefix}/matrix/$(date -u +%Y-
        %m-%dT%H-%M-%SZ)"
fi
```

New flags forwarded to `cmd/emulator-matrix`:

- `--concurrent N` → forwards to `--concurrent N`
- `--dev` → forwards to `--dev`
- `--tag <tag>` → consumed by the wrapper; does NOT forward (Containers binary has no concept of "tag")

G. `scripts/tag.sh` extension (Lava)

The Group A-prime `tag.sh` already requires `.lava-ci-evidence/<tag>/real-device-verification.{json,md}` to exist. Group B adds three gates that read the JSON:

```
attest_json=".lava-ci-evidence/${TAG}/real-device-
        verification.json"
[[ -f "$attest_json" ]] || die "missing $attest_json"

# Gate 1: reject if any row carries concurrent != 1
if jq -e '.rows[] | select(.concurrent != 1)' "$attest_json" >/
        dev/null; then
  die "tag refused: attestation contains concurrent != 1 rows
        (developer-iteration evidence cannot gate a tag)"
fi

# Gate 2: reject if run-level gating is false
if [[ "$(jq -r '.gating' "$attest_json")" != "true" ]]; then
  die "tag refused: attestation has gating: false (run was --
        concurrent or --dev)"
fi

# Gate 3: assert per-row diag.sdk == row.api_level
if jq -e '.rows[] | select(.diag.sdk != .api_level)'
        "$attest_json" >/dev/null; then
  bad=$(jq -r '.rows[] | select(.diag.sdk != .api_level) | "\
        (.name): claimed api=\(.api_level) but diag.sdk=\
        (.diag.sdk)"' "$attest_json")
  die "tag refused: per-row diag.sdk mismatches api_level:
$bad"
fi
```

The three gates are the mechanical anti-bluff teeth of Group B:

- **Gate 1** prevents a developer's iteration matrix from accidentally gating a release tag.
- **Gate 2** is the run-level safety net — even if Gate 1's per-row check somehow misses a case (e.g. the runner mis-records the row), the run-level `gating: false` flag rejects.
- **Gate 3** is the *forensic* gate — a row that claims it ran on API 34 but whose AVD reported `ro.build.version.sdk=33` is the canonical "AVD shadow" bluff. This gate makes that bluff unshippable.

H. Tests + bluff-hunt + closure attestation

Containers tests (added in `pkg/emulator/`):

Test	What it verifies
`TestKillByPort_NoMatch_NoOp`	KillByPort with no matching process

Test	What it verifies
	returns Matched=0, no kill calls issued
`TestKillByPort_StrictAdjacentMatch`	A process whose cmdline has `-port 5554` adjacent → matched; `-port 25554` adjacent → NOT matched
`TestKillByPort_SubstringSafety`	A process whose argv contains `25554` (no adjacent `-port`) does NOT match for port 5554
`TestKillByPort_RequiresSIGKILL_AfterGrace`	A process that survives SIGTERM gets SIGKILL after 5s; KillReport.Sigkilled contains the pid
`TestTeardown_FastPath_Succeeds`	After 30s adb-emu-kill grace expires, KillByPort matches and clears the emulator
`TestTeardown_FastPath_SkipOnMismatch`	KillByPort.Matched == 0 → Teardown returns the original timeout error (no other processes touched)
`TestParseJUnitFailures_AllPass_EmptySlice`	Well-formed JUnit XML with no failures → empty slice
`TestParseJUnitFailures_OneFailure_OneError_BothCaptured`	JUnit XML with 1 `<failure>` + 1 `<error>` → 2 entries with correct Type/ Message
`TestParseJUnitFailures_MalformedXML_SyntheticEntry`	Truncated XML → 1 synthetic `<unparseable>` entry; row not failed
`TestParseJUnitFailures_MultiTestsuite`	2 sibling `<testsuite>`

Test	What it verifies
	elements both contributing failures → all captured
`TestRun_Concurrent_AssignsUniquePorts`	--concurrent 3 → workers get ports 5554/5556/5558; each row records its assigned port
`TestRun_Gating_TrueOnDefaults`	--concurrent 1, --dev=false → RunReport.Gating == true
`TestRun_Gating_FalseOnConcurrent`	--concurrent 2 → RunReport.Gating == false
`TestRun_Gating_FalseOnDev`	--dev=true → RunReport.Gating == false

Every test MUST be falsifiable per Sixth Law clause 2 — the implementer's commit body records 5 mutation rehearsals (KillByPort strict-match dropped → TestKillByPort_StrictAdjacentMatch fails; SIGKILL grace skipped → TestKillByPort_RequiresSIGKILL fails; Teardown skip-on-mismatch dropped → TestTeardown_FastPath_SkipOnMismatch fails; JUnit `<error>` elements ignored → TestParseJUnitFailures_OneFailure_OneError fails; Gating defaulted to false → TestRun_Gating_TrueOnDefaults fails).

Lava parent tests (added under `tests/tag-helper/`):

Test	What it verifies
`test_tag_rejects_concurrent_attestation.sh`	tag.sh refuses an attestation whose any row has concurrent != 1
`test_tag_rejects_non_gating_attestation.sh`	tag.sh refuses an attestation with run-level gating: false
`test_tag_rejects_diag_sdk_mismatch.sh`	tag.sh refuses an attestation with diag.sdk != api_level on any row

Test	What it verifies
`test_tag_accepts_gating_serial_attestation.sh`	tag.sh accepts a clean serial attestation with all 3 gates green

The tests construct fixture JSON attestations + minimal git history and exercise tag.sh end-to-end (similar to the Group A-prime `tests/pre-push/check{4,5}_test.sh` harnesses).

Bluff-hunt evidence (`.lava-ci-evidence/bluff-hunt/2026-05-06-group-b-evening.json`):

Per §6.N.1.1 subsequent-same-day rule, a 1-2-file lighter incident-response hunt. Targets: 2 production files from the gate-shaping surface this group touched (cleanup.go::KillByPort + matrix.go::parseJUnitFailures). For each target: deliberate mutation, observed test failure, revert confirmation.

Closure attestation (`.lava-ci-evidence/Phase-Group-B-closure-2026-05-06-evening.json`):

The closure attestation records:

- The Containers branch HEAD SHA + 4-mirror SHAs
- The Lava parent commit SHA + 4-mirror SHAs
- Per-component check: each of A-H's commit SHA + tests count + bluff-hunt reference
- The 5 mutation rehearsals with observed failures + revert confirmations
- A pointer to the bluff-hunt JSON
- A "next group preview" line naming what Group C is candidate-scoped to cover (network simulation, hardware-screenshot capture, AVD image-cache management) — this is forward-looking only; no commitment.

4. Data Flow

4.1 Successful gating run (the canonical path)

```
operator: ./scripts/run-emulator-tests.sh
  └── auto-detect Lava-Android-X.Y.Z-NN
  └── EVIDENCE_DIR = .lava-ci-evidence/Lava-Android-X.Y.Z-NN/
matrix/<UTC>/
  └── invoke cmd/emulator-matrix --concurrent 1 (default)

cmd/emulator-matrix
  └── for each AVD (serial, 1 emulator at a time):
      ├── boot (cold-boot mandate, port 5554)
      ├── captureDiagnostic → DiagnosticInfo{target, sdk, device,
```

```
adb_devices_state}
      ├── install APK
      ├── run instrumentation
      ├── copy gradle.log (Group A-prime)
      ├── copy test-report/ (Group A-prime)
      ├── parseJUnitFailures → [] (all pass)
      ├── Teardown
      │     ├── adb -s emulator-5554 emu kill
      │     ├── wait 30s — exits cleanly → Teardown returns nil
      │     └── (KillByPort fast-path NOT invoked — emulator
already gone)
      └── append AVDRow{passed:true, diag, failure_summaries:[],
concurrent:1}
  └── write real-device-verification.json with gating: true
  └── write real-device-verification.md (human-readable mirror)
  └── exit 0

operator: scripts/tag.sh Lava-Android-X.Y.Z-NN
  ├── existing pretag-verify.sh (parser, mirror SHAs, etc.)
  ├── Gate 1: jq → no rows with concurrent != 1 → pass
  ├── Gate 2: jq → run-level gating == true → pass
  ├── Gate 3: jq → all rows diag.sdk == api_level → pass
  ├── existing 4-mirror push convergence verification
  └── tag created
```

4.2 Developer-iteration run (non-gating)

```
operator: ./scripts/run-emulator-tests.sh --concurrent 4 --dev
  └── EVIDENCE_DIR = .lava-ci-evidence/Lava-Android-X.Y.Z-NN/
matrix/<UTC>/
  └── invoke cmd/emulator-matrix --concurrent 4 --dev

cmd/emulator-matrix
  └── 4-worker pool, ports 5554/5556/5558/5560
  └── for each AVD: boot, capture, run, parse, Teardown
  └── write real-device-verification.json with:
        gating: false      ← --concurrent != 1 OR --dev
        rows[].concurrent: 4
  └── exit 0

operator: scripts/tag.sh Lava-Android-X.Y.Z-NN
  ├── Gate 1: jq → rows have concurrent: 4 → REJECT
  └── tag NOT created — operator must re-run gating matrix first
```

4.3 Forensic-divergence row (the bluff that Gate 3 catches)

```
attestation row: {name:"CZ_API34_Phone", api_level:34, diag:
{sdk:33, ...}}
  → CZ_API34_Phone is misconfigured (its system-image is
```

```
android-33)
  → matrix runner did NOT detect this — the AVD label said "API34"
  → Gate 3: jq → diag.sdk(33) != api_level(34) → REJECT
  → operator: fix the AVD config, re-run, attestation row's
diag.sdk
              matches api_level, tag passes
```

4.4 Stuck-emulator path (Teardown fast-path proves out)

```
cmd/emulator-matrix → Teardown(avd):
  ├── adb emu kill — issued
  ├── wait 30s — emulator process still in /proc (QEMU GPU lockup)
  ├── KillByPort(5554) →
  │     ├── /proc walk
  │     ├── matches 1 pid (QEMU with cmdline ... -port 5554 ...)
  │     ├── no other process matches (port 5556 pid is untouched)
  │     ├── SIGTERM → exits → Sigkilled list empty
  │     └── returns KillReport{Matched:1, Sigtermed:[12345]}
  └── re-poll 5s → emulator gone → Teardown returns nil
  → next AVD boots clean (no zombie left for cmd/emulator-cleanup)
```

5. Error Handling

Scenario	Behavior	Why
`KillByPort` /proc read fails on one pid	Skip that pid, continue walking	Best-effort; one unreadable /proc entry must not block the kill
`KillByPort` no match (Matched=0)	Return KillReport{Matched:0}, nil	Safety; concurrent emulators / sibling-project QEMUs untouched
`KillByPort` SIGTERM fails on a pid (process gone meanwhile)	Treat as success; remove from grace-poll list	Race between the /proc walk and the kill is benign
`KillByPort` SIGKILL fails	Append to KillReport.Sigkilled with negative marker; do NOT block return	Best-effort — the alternative is an

Scenario	Behavior	Why
		unbounded loop
Teardown fast-path runs, KillByPort matches but emulator still in /proc after 5s	Return error from Teardown; matrix runner records row failure	Honest signal — the emulator was unkillable; row is failure
`parseJUnitFailures` file missing	Return single synthetic entry `{class:"<unparseable>", message:"junit-xml read failed: ..."}`; row NOT failed by this	JUnit XML is evidence, not gating — Sixth Law clause 3
`parseJUnitFailures` malformed XML	Same as missing	Same — evidence corruption is non-fatal to the gating signal
`captureDiagnostic` getprop fails for one field	Leave that field zero/empty in DiagnosticInfo; continue	Partial diag is better than no diag
`captureDiagnostic` `adb devices -l` fails	Leave ADBDevicesState empty; continue	Same
Concurrent worker boot fails for 1 AVD	Row failure for that AVD; other workers continue	Bounded; the worker pool channel is drained either way
`--tag` and `--evidence-dir` both passed	`--evidence-dir` wins; warn on stderr	Existing flag retains existing semantics
`app/build.gradle.kts` missing or unparseable	Auto-detect returns "Lava-Android-unknown"; matrix continues	Iteration runs without a version do not crash; tag.sh's gates already reject this evidence anyway
`tag.sh` Gate 1/2/3 reject	Print the exact violating row(s); exit non-zero; tag NOT created	Anti-bluff teeth — refuse with diagnostic, not silently

6. Testing

Layer	What	Real-stack?	Falsifiability rehearsal?
`TestKillByPort_*` (Containers)	strict-adjacent matcher, no-op-on-mismatch, SIGKILL grace	injected procWalker + killer fakes (boundary mocks per Sixth Law)	YES — drop adjacency check → strict test fails
`TestParseJUnitFailures_*` (Containers)	encoding/xml decoder happy + error paths	real `os.ReadFile` + real `encoding/xml`	YES — drop `<error>` element handling → mixed-failure test fails
`TestRun_Concurrent_*` (Containers)	worker pool port assignment + Gating field	real goroutines + injected boot/teardown fakes	YES — default Gating to false → defaults test fails
`TestTeardown_FastPath_*` (Containers)	fallback timing, skip-on-mismatch safety	injected adb fake + injected KillByPort	YES — drop skip-on-mismatch → safety test fails
`tests/tag-helper/test_tag_rejects_*.sh` (Lava)	real tag.sh against fixture attestation JSON	real bash + real jq + real git fixture repo	YES — drop Gate 3 → diag-mismatch test fails
`tests/tag-helper/test_tag_accepts_*.sh` (Lava)	golden-path serial attestation passes all 3 gates	same	YES — flip Gate 1 to "reject if concurrent == 1" → accept test fails
Bluff-hunt (`.lava-ci-evidence/bluff-hunt/...`)	1-2 production files from gate-shaping surface	per §6.N.1.1 subsequent-same-day procedure	YES (mutation per file recorded)
Real-stack matrix run (operator)	`./scripts/run-emulator-tests.sh` against the 4-AVD minimum coverage from	YES — real cold-boot AVDs, real apk install, real instrumentation	NOT applicable to a single test — but the matrix run

Layer	What	Real-stack?	Falsifiability rehearsal?
	clause 6.I; produces a Group B attestation row for each AVD with diag + failure_summaries + concurrent		itself satisfies clause 6.I

7. Order of operations

Phase	Branch	Files	Commi
A. Containers code	Containers `lava-pin/2026-05-06-group-b`	`pkg/emulator/cleanup.go`, `pkg/emulator/cleanup_test.go`, `pkg/emulator/android.go`, `pkg/emulator/android_test.go`, `pkg/emulator/types.go`, `pkg/emulator/matrix.go`, `pkg/emulator/matrix_test.go`, `cmd/emulator-matrix/main.go`	1 comm (subage dispatc per Compon A→E wi two-sta review Group A prime pattern
B. Lava parent code	Lava `master`	`scripts/run-emulator-tests.sh`, `scripts/tag.sh`, `tests/tag-helper/test_tag_rejects_concurrent_attestation.sh`, `tests/tag-helper/test_tag_rejects_non_gating_attestation.sh`, `tests/tag-helper/test_tag_rejects_diag_sdk_mismatch.sh`, `tests/tag-helper/test_tag_accepts_gating_serial_attestation.sh`, `CLAUDE.md` (§6.I clause 4 doc extension)	1 comm
C. Pin bump + closure evidence	Lava `master`	`submodules/containers` SHA bump, `.lava-ci-evidence/bluff-hunt/2026-05-06-group-b-evening.json`, `.lava-ci-evidence/Phase-Group-B-closure-2026-05-06-evening.json`	1 comm

Total: **3 Lava parent commits + 1 Containers commit** (with the intra-phase fix-up commits the subagent-driven workflow may emit per the two-stage review). Mirror push to all 4 upstreams after each commit; 4-mirror SHA convergence verified via live `git ls-remote` (NOT stale `.git/refs/remotes/<r>/master` cache — Group A-prime forensics already recorded that lesson).

Estimated wall-clock: 6-8 hours (Phase A is the bulk; Phases B + C are mechanical once Phase A is green).

8. Constitutional bindings

- **Sixth Law clauses 1-5.** Every Group B test traverses the production code path the Lava operator's matrix run triggers. Falsifiability rehearsal recorded per test class. Primary assertion is on user-visible outcomes (KillByPort report, parsed FailureSummary, Gating bool, tag.sh exit code).
- **Sixth Law clause 6 (inheritance).** Every Group B file inherits the Anti-Bluff Pact recursively into Containers and Lava both.
- **Seventh Law clause 1 (Bluff-Audit stamp).** Every commit's body carries the test/mutation/observed/reverted block per §6.N.1.2 (which Group A-prime already enforces via pre-push Check 4).
- **Clause 6.I (Multi-Emulator Container Matrix).** Group B *strengthens* the matrix gate (Gate 3's diag.sdk == api_level enforcement) without changing the per-AVD attestation row schema's existing fields.
- **Clause 6.J (Anti-Bluff Functional Reality Mandate).** The three new tag.sh gates exist for one reason: prevent a tag from being cut against evidence that does not represent a real-user-completes-the-flow run. `--dev` and `--concurrent` exist precisely so a developer can iterate fast WITHOUT polluting the gating signal.
- **Clause 6.K (Builds-Inside-Containers Mandate) — debt status.** Group B does NOT close 6.K-debt (still owed; `pkg/vm/` for QEMU and the host toolchain → Containers extraction remain open). The matrix runner reliability fixes here are independent of the build-path discussion.
- **Clause 6.M (Host-Stability Forensic Discipline).** KillByPort is scoped to the matrix runner's own QEMU children — strict-adjacent match means no host-process collateral. Per the recorded Container- runtime safety analysis, podman-rootless cannot cause Class I host events; KillByPort's process-targeting widens nothing the runtime doesn't already permit.
- **Clause 6.N (Bluff-Hunt Cadence Tightening).** Phase C's bluff-hunt JSON is the §6.N.1.1 subsequent-same-day record. Group A-prime's pre-push Check 5 will mechanically validate it on push.
- **Decoupled Reusable Architecture.** Components A-E live in Containers because matrix-runner reliability + per-AVD diag + JUnit XML parsing
  - worker pool are the kind of capability the next vasic-digital consumer of `pkg/emulator/` would want identically. Components F + G live in Lava because the version-string detection and the tag.sh gates are Lava-domain.

- **Local-Only CI/CD.** No new hosted-CI surface. All gates are pre-push hooks + tag.sh + the operator's `./scripts/run-emulator-tests.sh` invocation.

9. Mirror policy

Per §6.C (mirror-state mismatch checks) and the Decoupled Reusable Architecture rule:

- Containers branch `lava-pin/2026-05-06-group-b` MUST converge at the same SHA on all 4 upstreams (github + gitflic + gitlab + gitverse) before the Lava parent's pin-bump commit references it.
- Lava parent commits (Phases B and C) MUST converge at the same SHA on all 4 upstreams before the closure attestation is finalized.
- Convergence verified via live `git ls-remote <upstream> <ref>` — NOT via the stale `.git/refs/remotes/<r>/master` cache. Group A-prime's reviewer iterations recorded the false-positive divergences this cache produces; do not repeat them.

10. Self-review checklist

Run by the spec author after writing this document, per the brainstorming skill's required checklist:

- ■ **Placeholder scan.** No "TBD", no "TODO", no "to be detailed later". Every component (A-H) has signature/algorithm/test detail.
- ■ **Internal consistency.** The architecture diagram, the components section, the data flow, the testing table, and the order-of-operations table all reference the same 5 in-scope items, the same 6+2 modified files (6 Containers + 2 Lava parent + tests/evidence), the same branch name (`lava-pin/2026-05-06-group-b`), the same gating-flip semantics (`--concurrent != 1 OR --dev` → `gating: false`).
- ■ **Scope check.** Single matrix-runner reliability + observability increment. Each of the 5 components is independently shippable but bundled because they share the same evidence schema (`AVDRow` extension) and the same tag.sh gate touches both `concurrent` and `gating`. Not too large for one plan.
- ■ **Ambiguity check.** Every "MUST" / "MAY" / "SHOULD" usage is intentional. The KillByPort safety property is stated three times in three contexts (component A, error handling,

constitutional bindings) precisely because the operator's "go with the safest approach, other processes may be used by other projects" instruction needs zero room for misreading.

Spec status: approved (operator, 2026-05-05 evening). Awaiting operator review of this written form before transitioning to `superpowers:writing-plans` for the implementation plan.